

08 · Flujo de datos

Documentacion del sistema

Version analizada: **0.1.0**

Commit analizado: **2282a33**

Fecha de generacion: **2026-08-27**

Generado a partir de 08-data-flow.md — los Markdown son la fuente unica.
No editar este PDF: editar el Markdown y regenerar.

Contenido de este documento

1. Vista general
2. Origen de los datos
3. Validación
4. Transformaciones
5. Almacenamiento
6. Quién consume cada dato
7. Datos que salen del equipo
8. Datos personales y sensibles procesados
9. Dónde puede haber pérdidas o inconsistencias
10. Trazabilidad de un hallazgo

De dónde sale cada dato, cómo se valida, en qué se transforma, dónde se guarda, quién lo consume y dónde puede perderse o quedar incompleto.

1. Vista general

Diagrama Mermaid #1 — código fuente (no renderizado en PDF)

```
flowchart TB
    subgraph ORIGEN["Origen"]
        A1["Utilidades de macOS<br/>ps · lsof · sptcl · csrutil · arp · codesign · log"]
        A2["Archivos del sistema<br/>plists de launchd · Info.plist de XProtect"]
        A3["SQLite ajeno<br/>TCC.db usuario y sistema"]
        A4["Sistema de archivos<br/>cachés y temporales"]
        A5["sysinfo<br/>procesos · memoria · interfaces"]
        A6["Estado propio<br/>config JSON · baselines · agent-state"]
    end
    end
    subgraph PARSE0["Parseo y validación"]
        B1["Parsers por superficie"]
    end
    end
    subgraph DECISION["Clasificación"]
        C1["classify_process · classify_entry<br/>classify_connection · classify_device"]
        C2["diff_surface · AnomalyTracker"]
        C3["build_alerts · derive_incident"]
    end
    end
    subgraph MODELO["Modelo"]
        D1["SystemSnapshot"]
    end
    end
    subgraph SALIDA["Consumo"]
        E1["GUI"]
        E2["CLI · stdout"]
        E3["JSON exportado"]
        E4["Reporte Markdown"]
        E5["SQLite propio"]
        E6["Notificación del sistema"]
        E7["Proveedor IA (opcional)"]
    end
    end

    A1 --> B1
    A2 --> B1
    A3 --> B1
    A4 --> B1
    A5 --> B1
    A6 --> C2
    B1 --> C1
    C1 --> C2
    C2 --> C3
    C3 --> D1
    D1 --> E1
    D1 --> E2
    D1 --> E3
    D1 --> E4
    D1 --> E5
    D1 --> E6
    E5 -.->|"solo el incidente resumido"| E7
```

La forma del grafo es la tesis del producto: **muchos orígenes, un solo modelo, muchas salidas**. Nada consume datos del sistema saltándose el modelo.

2. Origen de los datos

#	Origen	Mecanismo	Módulo	Frecuencia
1	Procesos, memoria, CPU, interfaces	Biblioteca <code>sysinfo</code>	<code>inspector.rs</code>	Cada captura
2	Usuario y línea de comandos por PID	<code>ps -axo pid=,user=,command=</code>	<code>macos.rs</code>	Cada captura

3	Conexiones de red	<code>lsof -i -n -P -FpLftPnT</code>	<code>macos.rs</code> → <code>network.rs</code>	Cada captura
4	Firma de código	<code>codesign -dvv <ruta></code>	<code>macos.rs</code>	Hasta 12 rutas por captura
5	Persistencia	Lectura de <code>.plist</code> + <code>crontab -l</code>	<code>launchd.rs</code>	Cada captura
6	Login items	<code>osascript</code> → System Events	<code>launchd.rs</code>	Solo bajo petición
7	Controles de seguridad	<code>spctl, csrutil, fdesetup, socketfilterfw, defaults, launchctl</code>	<code>security.rs</code>	Cada captura
8	Definiciones antimalware	Lectura de 4 <code>Info.plist</code>	<code>security.rs</code>	Cada captura
9	Permisos de privacidad	SQLite en solo lectura sobre dos <code>TCC.db</code>	<code>tcc.rs</code>	Cada captura
10	Vecinos de red	<code>arp -a -n, route, ifconfig</code>	<code>netscan.rs</code>	Cada captura
11	Barrido activo	$254 \times \text{ping}$ + <code>dscacheutil</code>	<code>macos.rs</code>	Solo bajo petición
12	Cachés y temporales	<code>WalkDir</code> sobre 7 raíces + <code>\$TMPDIR</code>	<code>temp_scan.rs</code>	Cada captura
13	Eventos de seguridad	<code>log show --predicate ...</code>	<code>macos.rs</code>	Solo bajo petición
14	Servicios de launchd	<code>launchctl list</code>	<code>launchd.rs</code>	Cada captura
15	Configuración	JSON local	<code>config.rs</code>	Al arrancar y al guardar
16	Baselines	SQLite propio	<code>persistence.rs</code>	Cada captura
17	Estado del agente	JSON local	<code>resilience.rs</code>	Al arrancar y en cada latido

3. Validación

El producto **no valida entradas de usuario** en el sentido clásico —no hay formularios ni API—, pero sí valida sistemáticamente lo que le devuelve el sistema operativo. Cinco técnicas, todas presentes en el código:

Técnica	Ejemplo	Efecto de un dato malo
Parseo tolerante con guardas	<code>let Ok(pid) = pid _text. parse() else { continue }</code>	Se salta esa línea
Descarte explícito de ruido	<code>(incomplete)</code> y <code>ff:ff:ff:ff:ff:ff</code> en ARP	La entrada no se crea
Detección de esquema	<code>PRAGMA table_ info(access)</code> en TCC	Se usa la columna correcta

Normalización antes de comparar	<code>normalize_mac, to_ascii_lowercase</code>	Evita falsos cambios
Valor por defecto seguro	<code>CodeSignature::Unknown, Severity::Warning</code> para «desconocido»	Nunca se asume confianza

Validación de la configuración: `serde` con `#[serde(default = "...")]` en cada campo. Un JSON parcial es válido; uno malformado hace que se usen los valores por defecto y se genere una advertencia visible.

Lo que **no** se valida:

- Rangos de los umbrales configurados. Nada impide poner `cpu_warning_percent = 500`; la consecuencia es que esa señal nunca dispara. **INFERENCIA**.
- Banderas desconocidas del CLI: se ignoran en silencio.
- La respuesta del proveedor IA se valida estructuralmente (forma esperada), pero su contenido es texto libre generado por un modelo.

4. Transformaciones

4.1 Cadena típica de un dato

Tomemos una conexión de red, de punta a punta:

Diagrama Mermaid #2 — código fuente (no renderizado en PDF)

```

flowchart LR
    A["lsof<br/>p501\ncGoogle Chrome\nnf35\nnn192.168.1.5:54321->142.250.1.1:443"]
    --> B["parse_lsof_field_output<br/>máquina de estados por campo"]
    --> C["split_endpoints<br/>local / remoto"]
    --> D["extract_ip + is_public_ip<br/>142.250.1.1 → pública"]
    --> E["classify_connection<br/>warning · 'Conexión saliente a IP pública'"]
    --> F["ConnectionInsight"]
    --> G["unique_public_remotes_by_pid<br/>agrupación por PID"]
    --> H["Heurística unusual-outbound<br/>si ≥ 4 destinos y proceso no habitual"]
    --> I["AnomalyEvent → Alert → IncidentSummary"]

```

Seis transformaciones desde una línea de texto hasta un incidente con evidencia. Ninguna vuelve a consultar el sistema.

4.2 Catálogo de transformaciones

Dato de origen	Transformación	Resultado
Bytes acumulados de E/S por PID	Resta contra la muestra anterior con <code>saturating_sub</code> ; primera muestra siembra	MB del intervalo
Bytes de memoria	<code>/ 1024³</code>	GB
Salida de <code>codesign</code>	<code>classify_codesign_output</code>	<code>CodeSignature</code>
Diccionario de un plist	<code>Program + ProgramArguments</code> según la semántica de launchd	Comando efectivo
<code>auth_value</code> de TCC	<code>decode_decision</code>	(texto, concedido)
Epoch de TCC	<code>format_epoch</code>	RFC 3339 o cadena vacía
MAC <code>00:11:22:...</code>	<code>normalize_mac</code>	<code>00:11:22:...</code>
IPv4	<code>subnet_prefix_of</code>	Prefijo <code>/24</code>

Prefijo de MAC	<code>vendor_from_mac</code> sobre 26 OUI	Fabricante o vacío
Fecha de un bundle	<code>(ahora - fecha).num_days().max(0)</code>	Antigüedad en días
Bytes de un árbol de directorios	<code>measure_directory</code> con tope	MB, nº de archivos, <code>truncated</code>
Métricas de proceso + firma	<code>classify_process</code>	Severidad, puntaje, motivos, categoría
Entrada de persistencia	<code>classify_entry</code>	<code>RiskLevel</code> y nota explicativa
Superficie completa	<code>*_watch_items</code>	<code>Vec<WatchedItem></code>
<code>WatchedItem</code> + baseline	<code>diff_surface</code>	<code>PersistenceChange</code>
Cambios + heurísticas	<code>build_alerts</code>	Alertas ordenadas y veredicto
Captura completa	<code>derive_incident</code>	<code>IncidentSummary</code> o <code>None</code>
Captura completa	<code>build_report</code>	Markdown de 11 secciones
Captura completa	<code>serde_json</code>	JSON exportable
Captura completa	<code>persist_snapshot</code>	Fila compacta en SQLite

4.3 Pérdida de información deliberada

Cada transformación descarta algo, y conviene saber qué:

Se pierde	Dónde	Por qué
Salida cruda completa de los comandos	Tras el parseo	Solo se guarda la primera línea útil como evidencia
Procesos no destacados	Al persistir	<code>snapshots</code> guarda solo el proceso dominante
Alertas más allá de <code>max_alerts</code>	<code>build_alerts</code>	Se truncan tras ordenar por severidad
Anomalías más allá de 5	<code>derive_incident</code>	<code>anomaly_events</code> guarda las cinco primeras
Conexiones más allá de 80	Solo en la impresión del CLI	La captura las conserva todas
Entradas de caché más allá de 40 000 por raíz	<code>measure_directory</code>	Se declara como medición aproximada
Incidentes duplicados consecutivos	<code>persist_incident</code>	Deduplicación por huella
Capturas más allá de 1 000	<code>trim_table</code>	Retención configurable

5. Almacenamiento

Dato	Destino	Duración	Formato
Resumen de captura	Tabla <code>snapshots</code>	1 000 filas	Columnas + <code>alerts_json</code>
Incidente	Tabla <code>incidents</code>	300 filas	Columnas + <code>payload_json</code>

Acción manual o de agente	Tabla <code>audit_log</code>	Indefinida	Columnas
Baseline de persistencia	Tabla <code>persistence_baseline</code>	Hasta aceptar otra	Columnas
Baselines de seguridad, TCC y red	Tabla <code>baseline</code>	Hasta aceptar otra	Columnas
Configuración	<code>rootcause-config.json</code>	Permanente	JSON
Estado del agente	<code>rootcause-agent-state.json</code>	Permanente	JSON
Captura exportada	<code>~/Downloads/rootcause-snapshot-<fecha>.json</code>	La que decida el usuario	JSON
Reporte	<code>~/Documents/RootCause/reports/...md</code>	La que decida el usuario	Markdown
Series de tendencia	Memoria (60 muestras)	Sesión	<code>Vec<f32></code>
Caché de firmas y deltas	Memoria	Sesión	<code>HashMap</code>

6. Quién consume cada dato

Consumidor	Qué recibe	Cómo
Sección Resumen	Veredicto, métricas, tendencias, alertas, salud del agente	<code>SystemSnapshot</code> completo
Sección Procesos	<code>processes</code> , filtrado por texto y severidad	Clonado del snapshot
Sección Conexiones	<code>connections</code>	Idem
Sección Red	<code>network</code> y, si se pidió, el escaneo profundo aparte	<code>NetworkScan</code>
Sección Persistencia	<code>persistence_entries</code> + <code>login_items</code> bajo demanda	Idem
Sección Seguridad	<code>security_controls</code> + <code>xprotect</code>	Idem
Sección Privacidad	<code>tcc</code>	Idem
Sección Almacenamiento	<code>caches</code>	Idem
Sección Historial	<code>history</code> , <code>incidents</code> , <code>audits</code> , <code>events</code>	Consultas a SQLite bajo demanda
CLI	La parte que pida el comando	<code>SystemSnapshot</code> o llamada directa al módulo
Reporte	Captura completa + hardware	<code>build_report</code>
Notificación del sistema	Título y detalle de la primera alerta crítica	<code>macos::notify</code>

Proveedor IA	Solo el incidente resumido	<code>build_payload</code>
--------------	----------------------------	----------------------------

7. Datos que salen del equipo

Este es el apartado corto y el más importante.

Salida	Ocurre por defecto	Qué contiene
Petición al proveedor IA	No. Requiere <code>ai.enabled = true</code> , <code>ai.endpoint</code> configurado y la variable de entorno con la clave	Título, tipo, resumen, hipótesis local y evidencia del incidente
Cualquier otra	—	No existe

No hay telemetría, ni comprobación de actualizaciones, ni envío de errores, ni analítica, ni servidor propio. `grep -rn "http" src/` solo devuelve la URL del repositorio en `meta.rs`, el endpoint configurable de IA y los `unwrap` de `curl` en `ai.rs`.

Lo que **no** viaja en la petición de IA, verificado por el test `el_payload_solo_lleva_el_incidente_resumido`: la captura completa, la lista de procesos, los permisos TCC, las rutas de cachés, el inventario de red y el historial.

8. Datos personales y sensibles procesados

Categoría	Ejemplos concretos	Dónde se procesa	Dónde se guarda
Identidad del usuario	Nombre de usuario, UID, nombre del equipo	<code>macos.rs</code>	Reporte (nombre de equipo); no en SQLite
Software instalado	Rutas de ejecutables, <code>Label</code> de plists, bundle ids con permisos	Todas las superficies	<code>persistence_baseline</code> , <code>baseline</code> , <code>payload_json</code>
Actividad de red	IP remotas, puertos, IP y MAC de vecinos	<code>network.rs</code> , <code>netscan.rs</code>	<code>baseline</code> , <code>payload_json</code>
Permisos concedidos	Qué app puede grabar pantalla, leer teclado o el disco entero	<code>tcc.rs</code>	<code>baseline</code> (solo sensibles concedidos)
Contenido de archivos	Ninguno. Se leen metadatos y plists de configuración, nunca documentos	—	—
Credenciales	Ninguna. No se leen llaveros ni almacenes de contraseñas	—	—

La lista de permisos TCC es probablemente el dato más sensible que el producto maneja: revela qué aplicaciones pueden observar al usuario. Se guarda solo en la baseline, en local, y nunca se envía.

9. Dónde puede haber pérdidas o inconsistencias

Punto	Qué puede pasar	Cómo se maneja	Residual
<code>lsof</code> sin root	Solo se ven los sockets del propio usuario	El CLI y la GUI lo declaran	Cobertura parcial de conexiones
<code>TCC.db</code> sin permiso	No hay datos de privacidad	<code>readable = false</code> + alerta	Sección vacía, declarada
Escaneo ARP pasivo	Solo aparecen equipos con los que ya se habló	Limitación declarada; <code>--deep</code> los despierta	Vecinos no vistos

Tope de 40 000 entradas	Medición de cachés aproximada	Limitación declarada por raíz truncada	Total infravalorado
Presupuesto de 12 firmas	Procesos sin firma resuelta	<code>signature = None</code> ; no se penaliza	Señal ausente, no falsa
Escritura en SQLite falla	La captura no entra al historial	Alerta de advertencia; la captura sí se muestra	Hueco en la serie
Configuración inválida	Se usan valores por defecto	Advertencia persistente en cada captura	Umbrales distintos a los esperados
Baseline borrada	Todo vuelve a parecer conocido	No hay protección	Pérdida real de capacidad de detección
Proceso muy corto	Nace y muere entre dos capturas	No se maneja	Invisible para el producto
<code>codesign</code> sobre binario sustituido	La caché de sesión mantiene la firma anterior	La caché muere con el proceso	Ventana de una sesión
Cambio de formato en una utilidad de macOS	El parser deja de reconocer el estado	<code>known = false</code> → «Desconocido» en amarillo	Pérdida de dato, no falso verde

La fila más importante es la penúltima categoría del producto en general: **RootCause solo ve lo que existe en el momento de la captura**. Un proceso que arranca, escribe y muere en menos de un intervalo de refresco no aparece en ninguna parte. Es una limitación estructural de un sensor basado en muestreo, y el producto no la disimula.

10. Trazabilidad de un hallazgo

Ejemplo real, seguido por el código: **aparece un LaunchDaemon nuevo que ejecuta un binario sin firmar desde `/tmp`**.

Diagrama Mermaid #3 — código fuente (no renderizado en PDF)

```
sequenceDiagram
    participant FS as /Library/LaunchDaemons
    participant L as launchd.rs
    participant M as macos.rs
    participant I as inspector.rs
    participant P as persistence.rs
    participant A as anomaly.rs
    participant R as rules.rs
    participant U as Usuario

    FS->>L: com.fake.helper.plist
    L->>L: parse_launch_plist → command = /tmp/.helper
    L->>M: code_signature("/tmp/.helper")
    M->>L: Unsigned
    L->>L: classify_entry → 26+30+25+30 = 111 → Critical
    L->>I: PersistenceEntry
    I->>P: load_persistence_baseline()
    P->>I: baseline sin esa clave
    I->>I: change_status = Added
    I->>A: persistence_change_event()
    A->>I: AnomalyEvent kind=persistence-change, severity=Critical (escalado)
    I->>R: build_alerts()
    R->>I: Alert crítica en primera posición + veredicto Critical
    I->>R: derive_incident()
    R->>I: IncidentSummary con evidencia y acciones
    I->>P: persist_incident() · persist_snapshot()
    I->>U: semáforo rojo, alerta, sección Persistencia marcada NUEVA
```

Cada flecha es una llamada real y comprobable. El puntaje 111 sale de sumar: ámbito `GlobalDaemon` (26) + ruta temporal (30) + binario oculto (25) + sin firma (30).

Siguiente lectura recomendada: [09 · APIs e integraciones](#).